

8.4 - Anti-corruption Layer e Branch by Abstraction

Proteger o novo design do legado

Quando o legado contamina o novo

O risco de herdar o problema que se quer abandonar

- **Sistema novo e legado precisam conversar durante a migração**
 - Cada chamada ao legado traz o modelo dele junto
- **Sem fronteira, o modelo legado vaza para o novo**
 - O novo acaba replicando as estruturas que queria abandonar
- **A modernização perde o sentido se herda o problema**
 - Você troca de tecnologia mas mantém o mesmo modelo ruim
- **A solução é isolar os dois mundos por contrato**
 - Dois padrões resolvem isso: ACL e Branch by Abstraction

Quarenta tabelas tentando invadir o novo

- **Novo serviço de rotas usa modelo de domínio limpo**
 - Monólito legado tem modelo relacional de 2021

LUNALOG

O novo serviço de rotas da LunaLog usa um modelo de domínio limpo, orientado a eventos. O monólito legado usa um modelo relacional normalizado de 2021, com quarenta tabelas interligadas. Sem um Anti-corruption Layer, esse modelo legado contaminaria o design do serviço novo, forçando o time a replicar no sistema novo exatamente as estruturas problemáticas que estava tentando abandonar. A camada de tradução é o que mantém os dois mundos separados.

Anti-corruption Layer: a camada de tradução

Uma fronteira entre dois modelos de mundo

- **ACL é uma camada que traduz entre os dois modelos**
 - Converte dados do legado no modelo limpo do novo
- **O novo nunca enxerga as estruturas do legado**
 - Toda a tradução fica concentrada em um único lugar
- **Quando o legado morre, você descarta só a camada**
 - O domínio novo permanece intacto e sem cicatrizes
- **O custo é manter o tradutor enquanto os dois coexistem**
 - Vale a pena para proteger o ativo mais valioso, o domínio

Branch by Abstraction

- **Branch by Abstraction troca um componente interno sem forks**
- **Cria uma abstração na frente do componente antigo**

Você implementa o componente novo atrás da mesma abstração, alterna a implementação por configuração e remove a antiga quando tudo migrou. Tudo acontece na branch principal, sem código paralelo divergindo por meses.

- Ideal para substituir peças grandes dentro do mesmo sistema

Qual padrão usar em cada caso

Anti-corruption Layer

- Fronteira entre dois sistemas diferentes
- Protege o novo do modelo do legado
- Traduz dados que cruzam a fronteira
- Descartado quando o legado morre

Branch by Abstraction

- Substituição interna no mesmo sistema
- Troca um componente grande por outro
- Alterna implementações por configuração
- Tudo na branch principal, sem fork



Com ACL e Branch by Abstraction você protege o design do sistema novo. Mas modernizar a aplicação sem tocar nos dados é modernizar metade do problema. A outra metade, os dados históricos, continua presa no banco legado. Como movê-los sem janela de manutenção, sem perder nada e com rollback se algo falhar?

Migração de dados sem downtime